Bridge Design Pattern

Category: Structural | **GoF Reference:** Gang of Four — Design Patterns (1994), p. 151

Table of Contents

1. Intent

The Bridge pattern **separates an abstraction from its implementation** so that the two can vary independently.

The word "bridge" is deliberate: the pattern literally builds a bridge between two independent class hierarchies — the abstraction hierarchy and the implementation hierarchy — and connects them through **composition rather than inheritance**. Each hierarchy can grow and change without affecting the other.

In classical inheritance, you extend a base class to add behavior, and you extend it again to add a variant of that behavior. This works for one dimension of variation. The moment you have *two* orthogonal dimensions (for example, *shapes* and *colors*, or *UI controls* and *rendering engines*, or *payment flows* and *payment gateways*), inheritance starts generating a combinatorial explosion of subclasses. Bridge cuts that explosion down by making one hierarchy *hold a reference to* the other instead of inheriting from it.

The core idea: **"Prefer composition over inheritance."** Bridge is one of the purest expressions of this principle in the GoF catalog.

---

2. The Problem It Solves — Class Explosion

Suppose you are building a drawing library. You have shapes and colors. Without Bridge, you model every combination as its own class:

```
RedTriangle BlueTriangle GreenTriangle RedCircle BlueCircle GreenCircle RedSquare
BlueSquare GreenSquare
```

With 3 shapes and 3 colors you already have **9 classes**. Add a fourth color (Yellow) and you must add 3 more classes. Add a fourth shape (Pentagon) and you must add 4 more classes. The formula is:

```
Total classes = N shapes × M colors
```

At 10 shapes and 10 colors: **100 classes**. At 20 shapes and 20 colors: **400 classes**. The codebase grows quadratically. Each new class mostly duplicates logic from its siblings. Tests multiply. Every refactor touches dozens of files.

Bridge collapses this to:

```
Total classes = N shapes + M colors
```

At 10 shapes and 10 colors: **20 classes**. At 20 shapes and 20 colors: **40 classes**. The codebase grows linearly. Adding a new color is one file. Adding a new shape is one file. Neither touches the other.

The mechanism: instead of `RedTriangle extends Triangle`, you write `Triangle` with a field of type `Color`. The triangle delegates color-related behavior to whichever `Color` instance it holds at runtime. This is the bridge.

---

3. When to Use

Use the Bridge pattern when:

> **You want to avoid a permanent binding between abstraction and implementation.** If the implementation should be selectable or switchable at runtime — for example, choosing a

database driver at startup or selecting a rendering backend based on the OS — Bridge lets you inject any conforming implementation without changing the abstraction.

Both the abstraction and its implementation should be extensible through subclassing, independently. If you have two separate reasons a class might grow (e.g., more shapes vs. more colors), those two reasons belong in two separate hierarchies, not one tangled one.

Changes to the implementation should not impact client code. Clients program to the abstraction (`Shape`). The `Color` interface can change its internals, gain new implementations, or be swapped entirely — clients never recompile or change.

You have a class explosion caused by multiple dimensions of variation. Whenever you find yourself naming classes `XYZ` where X varies along one axis and Z varies along another (e.g., `MySQLUserRepo`, `PostgresUserRepo`, `MySQLOrderRepo`, `PostgresOrderRepo`), Bridge is the remedy.

You need to share an implementation among multiple objects and this fact should be hidden from the client. A single `Color` instance can be shared by many shapes. The client uses `Shape`, never knowing whether the color object is shared or exclusive.

You are working across platform boundaries. Bridge is especially natural when the abstraction lives in portable, platform-independent code (your business logic) and the implementation lives in platform-specific code (OS calls, vendor SDKs, hardware drivers).

---

4. Structure — ASCII UML

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Shape
(Abstraction) ■ ■ Color (Implementor) ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ # color : Color ■◆■■■■■■■■ + fill() :
String ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ + draw() : String (abstract)■ ▲ ▲ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ▲
■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■ ■ Red ■ ■ Blue ■ ■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■ Triangle ■ ■+ fill() ■ ■+ fill() ■ ■
(RefinedAbstraction■ ■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■+
draw() : String ■ ■■■■■■■■■■■■■■■■■■■■■■■■■ ◆ = composition ("has-a") ▲ = inheritance
("is-a") The ◆ arrow is the bridge.
```

Roles in this diagram:

Role	Class in this module	Responsibility

Abstraction	`Shape`	Defines the high-level interface; holds the implementor reference
RefinedAbstraction	`Triangle`	Extends the abstraction; adds specific shape logic
Implementor	`Color`	Defines the low-level interface the abstraction delegates to
ConcreteImplementor	`Red`, `Blue`	Provides a concrete rendering of the implementor interface

The critical structural point: `Shape` *contains* `Color` through composition (the ◆ arrow). This is the bridge. `Triangle` does not extend `Red`; it holds a `Color` and calls `color.fill()` when drawing. The two hierarchies — the shape hierarchy on the left and the color hierarchy on the right — are completely decoupled from each other.

---

5. Package Structure Explained

```
com.design.patterns.bridge ■ ■■■ BridgeDesignPattern.java (driver / main class) ■ ■■■ contract/ ■■■ Color.java (Implementor interface) ■■■ Shape.java (Abstraction abstract class) ■ ■■■ concrets/ ■■■ Red.java (ConcreteImplementor) ■■■ Blue.java (ConcreteImplementor) ■■■ Triangle.java (RefinedAbstraction)
```

Why `contract` holds both interfaces?

The `contract` package groups the *stable, defining contracts* of the pattern — the parts that change least. `Color` is the implementor interface; `Shape` is the abstraction. Both define APIs (contracts) that the rest of the codebase depends on. Placing them together signals: "these are the anchors; extensions live elsewhere." Client code that wants to program against abstractions only needs to import from `contract`.

Why `concrets` is a sub-package of `contract`?

The `concrets` sub-package signals: "these are concrete realizations of the contracts defined one level up." Keeping them inside `contract` preserves locality — you can find the interface and all its implementations by navigating a single package tree — while still separating the stable API (`contract`) from the volatile implementations (`contract.concrets`). The sub-package structure also

mirrors the dependency direction: `concrets` depends on `contract`, never the reverse.

Why are both `Red`/`Blue` (color implementations) and `Triangle` (shape implementation) in the same `concrets` package?

In this pedagogical implementation, both dimensions are concrete extensions, so they share one `concrets` package. In a production codebase you would likely split them further: `contract.colors` and `contract.shapes`, each holding their respective implementations.

---

6. Line-by-Line Explanation of Every File

6.1 `Color.java` — The Implementor Interface

```
package com.design.patterns.bridge.contract;
```

Declares that this type belongs to the `contract` package — the stable API layer. All classes that depend on the `Color` contract import from here. The package path encodes the context: this is a bridge pattern contract inside the broader design-patterns project.

```
public interface Color {
```

Declares `Color` as a Java interface, not a class. An interface enforces a contract without providing implementation, making it the correct choice for the implementor role. Any class that wants to be a color in this system must implement this interface. The name `Color` is intentional: it describes the *concept*, not any specific color.

```
String fill();
```

The single method of the implementor interface. It takes no parameters and returns a `String` representing the color's name or value. The method is named `fill` because it conceptually "fills" a shape with a color. Returning a `String` (rather than printing directly or returning `void`) makes the method pure and testable — it produces output that callers can use, log, or assert against. Every concrete color class must provide an implementation of this method.

```
}
```

Closes the interface declaration. The interface is intentionally minimal — one method, one responsibility. This follows the Interface Segregation Principle: clients depend only on what they actually use.

6.2 `Shape.java` — The Abstraction

```
package com.design.patterns.bridge.contract;
```

Same package as `Color`. Both contracts live together in `com.design.patterns.bridge.contract`. This is the only import `Shape` needs from within its own layer.

```
public abstract class Shape {
```

`Shape` is an abstract class, not an interface. This choice is deliberate and important (discussed further in Section 10). Abstract class means: `Shape` is partially implemented — it holds the `color` field and the constructor — but leaves `draw()` for subclasses to complete. The `abstract` keyword prevents instantiation of `Shape` directly; you must subclass it and provide `draw()`.

```
protected final Color color;
```

This single field is the **bridge**. `Shape` does not extend `Color`; it *holds* a `Color`. The `protected` modifier allows subclasses (`Triangle`, and future shapes) to read `color` directly without needing a getter, which keeps the code tight. The `final` modifier ensures the color reference cannot be reassigned after construction — a shape's color identity is fixed at birth. `Color` is the interface type, not a concrete type: `Shape` knows only that it holds something that can `fill()`, not whether that something is `Red`, `Blue`, or any other color.

```
protected Shape(Color color) {
```

The constructor accepts a `Color` and stores it. It is `protected` so that only subclasses and classes in the same package can call it — external code cannot instantiate `Shape` directly (nor would they want to, since it is abstract). The parameter name `color` shadows the field name, which is resolved by `this.color = color` on the next line.

```
this.color = color;
```

Assigns the injected `Color` implementation to the field. This is **constructor injection** — the dependency (`Color`) is injected through the constructor rather than created inside the class. This makes `Shape` independent of any specific color; it works with any object that satisfies the `Color` contract, including mocks in tests.

```
abstract public String draw();
```

Declares the abstract method that all concrete shapes must implement. `String` return type (parallel to `Color.fill()`) keeps the method pure. Every shape has its own way of describing itself visually, but all shapes expose the same method signature — `draw()` — so client code can call `draw()` on any `Shape` without knowing its concrete type.

```
}
```

Closes the abstract class.

6.3 `Red.java` — Concrete Implementor

```
package com.design.patterns.bridge.contract.concrets;
```

Located in the `concrets` sub-package. This is a concrete class — an implementation — so it lives below the `contract` layer.

```
import com.design.patterns.bridge.contract.Color;
```

Imports the `Color` interface that this class implements. The dependency direction is correct: `concrets` depends on `contract`, not the other way around.

```
public class Red implements Color {
```

`Red` is a concrete class that provides one implementation of the `Color` contract. The `implements Color` declaration makes `Red` a valid substitution wherever a `Color` is expected — including the `Shape` constructor.

```
@Override
```

Standard Java annotation asserting that the method below overrides or implements a method from a supertype. This causes a compile-time error if the method signature does not actually match any method in `Color`, which guards against typos or interface changes that break implementations silently.

```
public String fill() {
```

The concrete implementation of the `fill()` method declared in `Color`. This is the body of the bridge's right-hand side for the red case.

```
return "RED";
```

Returns the string `"RED"` (uppercase). This is the simplest possible implementation — a plain string literal. In a real application, this might return an ANSI color code, an RGB hex value, or a localized color name.

```
} }
```

Closes the method and the class.

6.4 `Blue.java` — Concrete Implementor

```
package com.design.patterns.bridge.contract.concrets;
```

Same package as `Red`. Both concrete colors live together.

```
import com.design.patterns.bridge.contract.Color;
```

Same import as `Red`. Both implementors depend on the same `Color` interface.

```
public class Blue implements Color {
```

A second concrete implementor. `Blue` and `Red` are siblings — both implement `Color`, both are interchangeable anywhere a `Color` is needed. Neither knows about the other. Neither knows about `Shape`.

```
@Override public String fill() { return "Blue";
```

Note the inconsistency with `Red`: `Red.fill()` returns `"RED"` (all caps) while `Blue.fill()` returns `"Blue"` (mixed case). This is a minor stylistic inconsistency in the implementation — in production code you would standardize the return value format. The pattern itself is unaffected.

```
} }
```

Closes the method and the class.

6.5 `Triangle.java` — Refined Abstraction

```
package com.design.patterns.bridge.contract.concrets;
```

Triangle is a concrete shape — a refined abstraction — so it lives in `concrets` alongside the concrete colors.

```
import com.design.patterns.bridge.contract.Color;
```

Triangle's constructor needs `Color` as a parameter type to pass to `super()`. This import brings in the `Color` interface.

```
import com.design.patterns.bridge.contract.Shape;
```

Triangle extends `Shape`. This import brings in the abstract base class.

```
public class Triangle extends Shape {
```

`Triangle` is a concrete, instantiable class that extends the abstract `Shape`. It is the "Refined Abstraction" in Bridge terminology: it extends the abstraction (`Shape`) by providing a concrete `draw()` implementation for triangles specifically.

```
public Triangle(Color color) {
```

Triangle's constructor accepts a `Color` argument. The constructor is `public` — external code like the driver class needs to instantiate `Triangle` directly. The parameter type is `Color` (the interface), not `Red` or `Blue`, so `Triangle` is decoupled from any specific color. You can construct `new Triangle(new Red())`, `new Triangle(new Blue())`, or `new Triangle(anyColorYouInvent)`.

```
super(color);
```

Delegates to `Shape`'s protected constructor, which stores the `Color` reference in the `protected final Color color` field. `Triangle` itself does not have a `color` field — it inherits the field from `Shape`. The `super(color)` call is mandatory: Java requires a call to the superclass constructor as the first statement in a subclass constructor, and `Shape` has no no-arg constructor, so this delegation is explicit and unavoidable.

```
}
```

Closes the constructor. Triangle is fully initialized after this — it holds a `color` (via the inherited field) and is ready to `draw()`.

```
@Override public String draw() {
```

The concrete implementation of the abstract method declared in `Shape`. This is where Triangle defines its specific behavior. The `@Override` annotation confirms this satisfies the contract.

```
return "Triange shape with color : " + color.fill();
```

This line is the bridge **in action**. Triangle calls `color.fill()` — it delegates the color-related behavior to the `Color` implementor it was given at construction time. Triangle does not know if `color` is `Red`, `Blue`, or anything else. It just calls `fill()` and incorporates the result into its own output string. Note there is a typo in the source: `"Triange"` should be `"Triangle"`. The concatenation `"Triange shape with color : " + color.fill()` builds the complete result from triangle's own part and the color's part.

```
} }
```

Closes the method and the class.

6.6 `BridgeDesignPattern.java` — The Driver

```
package com.design.patterns.bridge;
```

The driver lives in the root package of the module, one level above `contract`. It is the entry point, not a pattern participant.

```
import com.design.patterns.bridge.contract.Shape;
```

The driver imports the `Shape` abstraction — the left side of the bridge. The driver programs to the abstraction, not to the concrete `Triangle` class (though it must use `Triangle` in the `new` expression to instantiate).

```
import com.design.patterns.bridge.contract.concrets.Red;
```

Imports the `Red` concrete implementor — one specific color. The driver knows about `Red` only at the point of object construction. Once constructed, the rest of the code works through `Color` and `Shape` interfaces.

```
import com.design.patterns.bridge.contract.concrets.Triangle;
```

Imports the `Triangle` refined abstraction. Again, the driver knows about `Triangle` for instantiation, but after that treats it as a `Shape`.

```
public class BridgeDesignPattern {
```

The driver class, named after the pattern. It is a plain class with a static `main` — no inheritance, no pattern participation.

```
public static void main(String[] args) {
```

The standard JVM entry point. This is the only method in the class.

```
System.out.println("Bridge Design Pattern");
```

Prints the pattern name as a header, confirming the program launched and identifying the output visually.

```
Shape triangle = new Triangle(new Red());
```

The key line of the demo. Three things happen here:

- `new Red()` creates a concrete `Color` implementor — the right side of the bridge.
- `new Triangle(new Red())` creates a concrete `Shape` (refined abstraction), injecting `Red` through the constructor — this is the bridge being assembled.
- `Shape triangle` stores the result as the abstract type `Shape` — from this line forward, the driver calls `draw()` on `triangle` without knowing or caring that it is a `Triangle` holding a `Red`.

The power of Bridge is visible in this single line: you could write `new Triangle(new Blue())` to get a blue triangle, or `new Circle(new Red())` when you add a `Circle` class, and nothing else changes.

```
System.out.println(triangle.draw());
```

Calls `draw()` on the `Shape` reference. Polymorphism dispatches to `Triangle.draw()`. The output is the return value of `Triangle.draw()`, which internally called `Red.fill()`. The driver never called `fill()` directly — it had no knowledge of `Color` at this point.

```
} }
```

Closes the method and the class.

7. Execution Flow

Here is a step-by-step trace of exactly what happens from the moment the JVM calls `main()` to the moment a string is printed:

```
1. JVM calls BridgeDesignPattern.main(String[]) 2. System.out.println("Bridge Design Pattern") → Prints: "Bridge Design Pattern" 3. new Red() → JVM allocates a Red instance on the heap. → Red has no fields; no constructor logic runs beyond Object(). → Returns a Red reference. 4. new Triangle(new Red()) → JVM allocates a Triangle instance on the heap. → Triangle(Color color) constructor is called with the Red instance. → Inside Triangle(): super(color) is called. → Inside Shape(Color color): this.color = color → The `color` field of the Shape portion of Triangle is set to the Red instance. → Triangle constructor returns. → Returns a Triangle reference (stored as Shape triangle). 5. triangle.draw() → Polymorphic dispatch: runtime type is Triangle, so Triangle.draw() is called. → Inside Triangle.draw(): "Triange shape with color : " + color.fill() → color.fill() is called on the Red instance. → Inside Red.fill(): return "RED" → Returns "RED" to Triangle.draw(). → Concatenation produces: "Triange shape with color : RED" → Triangle.draw() returns "Triange shape with color : RED" 6. System.out.println("Triange shape with color : RED") → Prints: "Triange shape with color : RED" 7. main() returns. JVM exits.
```

The bridge crossing happens at step 5: `Triangle.draw()` calls across to `Red.fill()`. `Triangle` and `Red` are in separate hierarchies, completely independent — they communicate only through the `Color` interface contract. You could replace `Red` with any `Color` implementation and step 5 would produce a different string without any change to steps 1–4 or step 6.

---

8. Real-World Use Cases

8.1 GUI Frameworks — Rendering API vs. UI Controls

GUI toolkits like Java's AWT/Swing or Qt face a classic Bridge scenario. There are two independent axes of variation:

- **UI Controls:** Button, TextField, CheckBox, ComboBox, ...
- **Rendering Backends:** Windows GDI, macOS Quartz, Linux X11, OpenGL, ...

Without Bridge: `WindowsButton`, `MacButton`, `LinuxButton`, `WindowsTextField`, `MacTextField`, ... Dozens of classes, each mixing UI logic with platform rendering code.

With Bridge: `Button` holds a reference to a `Renderer` interface. `WindowsRenderer`, `MacRenderer`, `OpenGLRenderer` implement `Renderer`. Adding a new control type adds one class. Adding a new platform adds one renderer. The two hierarchies evolve independently.

8.2 Database Drivers — JDBC Abstraction vs. Vendor Implementation

Java's JDBC is a textbook Bridge. Application code uses `java.sql.Connection`, `Statement`, `ResultSet` — these are the abstraction layer. MySQL Connector/J, PostgreSQL JDBC, Oracle Thin Driver — these are the implementations. Your application never touches MySQL-specific classes; it holds a `Connection`. Switching databases is swapping the `Driver` registration at startup; no application code changes.

8.3 Logging Frameworks — Logger API vs. Appender

SLF4J is a Bridge. Application code calls `Logger.info(...)` — the abstraction. Logback, Log4j2, java.util.logging — these are implementations plugged in at runtime via the `LoggerFactory` binding. Your application compiles against `org.slf4j:slf4j-api`. Which backend runs is a deployment decision. Adding a new log destination (Splunk, Datadog) means adding a new Appender implementation; application code is untouched.

8.4 Payment Processing — Payment Flow vs. Payment Gateway

An e-commerce platform has two axes:

- **Payment flows:** OneTimeCharge, Subscription, Installment, Refund
- **Payment gateways:** Stripe, PayPal, Razorpay, BankTransfer

Without Bridge: `StripeOneTimeCharge`, `PayPalOneTimeCharge`, `RazorpaySubscription`, ... — N×M classes, each pairing a flow with a gateway.

With Bridge: `PaymentFlow` (abstraction) holds a `PaymentGateway` (implementor). `Subscription extends PaymentFlow` handles recurring logic; `StripeGateway implements PaymentGateway` handles Stripe API calls. Adding PayPal requires one class. Adding a Layaway flow requires one class. Neither touches the other.

8.5 Device Drivers — OS Abstraction vs. Hardware Implementation

Operating systems use Bridge to manage hardware. The kernel defines an abstract device interface: `read()`, `write()`, `ioctl()`. Hardware vendors implement that interface for their specific chips. A USB storage driver and a network card driver both conform to the same kernel interface. The OS "abstraction" (file system, network stack) calls the standard interface; hardware vendors provide the "implementors." Adding a new graphics card model requires a new driver implementation; the kernel is unaffected.

---

9. Bridge vs Adapter

These two patterns are frequently confused because both involve two class hierarchies communicating through a reference. The critical difference is *when* and *why* they are applied.

Dimension	Bridge	Adapter
Purpose	Design for extensibility upfront	Make two incompatible interfaces work together retroactively
When designed	Before implementation — part of the initial architecture	After the fact — one class already exists with an incompatible interface
Relationship	Abstraction owns the implementor reference from day one	Adapter wraps an existing class to satisfy a new interface
Direction	Both hierarchies designed together with the bridge in mind	Adapter is a one-way adapter: it translates to a pre-existing target
Intent	"These two things should vary independently"	"I need to use this existing class, but it doesn't match the interface I need"

Concrete example of the distinction:

- Bridge (this module): `Shape` and `Color` are designed together. `Shape` is written knowing it will hold a `Color`. The bridge is intentional from the start.
- Adapter: You have a legacy `LegacyColorProvider` class with a method `getColorName()`. You need it to work as a `Color` (which requires `fill()`). You write a `LegacyColorAdapter implements Color` that wraps `LegacyColorProvider` and delegates `fill()` to `getColorName()`. You never changed `LegacyColorProvider`; you adapted it.

Think of it this way: Bridge is **architectural** (planned), Adapter is **tactical** (after-the-fact fix).

---

10. Key Design Decisions

Why `protected final Color color`?

Three separate modifiers, each with a distinct purpose:

`protected`: The `color` field needs to be accessible in `Triangle.draw()` without a getter method. Since `Triangle` is a subclass of `Shape`, `protected` grants access. Making it `private` would force a getter, adding boilerplate. Making it `public` would expose the internal implementor reference to all client code, breaking encapsulation.

`final`: A shape's color is set at construction and should not change. `final` enforces this at compile time — no code inside `Shape` or any subclass can reassign `color` after the constructor returns. This makes `Shape` instances easier to reason about: once built, their color contract never changes.

`Color` **interface type, not concrete**: `Shape` holds a reference of type `Color`, not `Red` or `Blue`. This is what makes the bridge work — `Shape` is decoupled from all specific colors. You can pass any `Color` implementation at construction time, now or in the future, without modifying `Shape`.

Why `abstract class Shape` instead of `interface Shape`?

An interface cannot hold instance fields or provide constructor logic. The bridge requires `Shape` to store a `Color` reference in `protected final Color color` and initialize it via `protected Shape(Color color)`. Neither is possible in an interface.

An abstract class is the right tool when you need to:
1. Define abstract methods that subclasses must implement (`draw()`).
2. Provide concrete state shared by all subclasses (`color` field).
3. Enforce an initialization protocol via a constructor (`this.color = color`).

An interface would force each subclass (`Triangle`, `Circle`, etc.) to independently declare and initialize its own `color` field, duplicating the bridge setup in every concrete shape. Abstract class centralizes that boilerplate once.

Why `super(color)` in the Triangle constructor?

`Shape` declares no no-arg constructor. When you subclass a class that has no default (no-arg) constructor, Java requires that your subclass constructor explicitly call a superclass constructor as its first statement. `super(color)` invokes `Shape(Color color)`, which stores the `Color` reference.

Beyond the mechanical requirement, `super(color)` is correct by design: the bridge connection (`this.color = color`) belongs in the base abstraction (`Shape`), not in each refined abstraction. Every shape needs a color stored the same way. Centralizing that in `Shape`'s constructor and delegating to it via `super()` is the DRY (Don't Repeat Yourself) expression of the pattern.

Why does `String fill()` return a value instead of `void`?

A `void fill()` would be forced to produce output as a side effect — either printing directly (`System.out.println`) or writing to some shared mutable state. Side-effecting implementors are:

- **Harder to test**: You cannot assert on a return value; you must intercept stdout or check shared state.
- **Less composable**: `Triangle.draw()` cannot build a string that *includes* the color; it could only trigger the color to print separately, losing the ability to compose the output.
- **Inflexible**: The caller (Triangle) loses control over what happens with the color information. It cannot log it, return it to a higher caller, or format it differently.

Returning `String` keeps `fill()` a **pure function**: given no input, it returns a predictable value with no side effects. `Triangle.draw()` can then compose the full output string (`"Triange shape with color : " + color.fill()`) and return it, leaving the decision of what to do with that string to the caller. The driver (`BridgeDesignPattern`) then decides to print it.

---

11. Output

Running `BridgeDesignPattern.main()` produces:

```
Bridge Design Pattern Triange shape with color : RED
```

Note: `"Triange"` is a typo in `Triangle.draw()` — it should read `"Triangle"`. The pattern is fully correct; only this string literal has a spelling error.

To see a blue triangle instead, change one line in the driver:

```
Shape triangle = new Triangle(new Blue());
```

Output becomes:

```
Bridge Design Pattern Triange shape with color : Blue
```

No other file changes. No recompilation of `Shape`, `Color`, `Red`, or any contract. This is Bridge in practice: the driver assembles the bridge at construction time, and the rest flows through interfaces.